Deep Learning Fundamentals

These notes cover the core ideas behind training neural networks: how a network represents a function, how we measure error, and how gradients are used to learn.

Neurons and layers

A neuron computes a weighted sum of its inputs, adds a bias, and passes the result through a non-linear activation function such as ReLU, sigmoid or tanh. Stacking layers of neurons lets the network compose simple functions into complex ones. Without non-linear activations, any number of stacked layers collapses into a single linear map.

ReLU (rectified linear unit) outputs max(0, x). It is cheap to compute and avoids the vanishing-gradient problem that sigmoid and tanh suffer from in deep networks, because its gradient is exactly 1 for positive inputs. Its downside is "dead neurons": units whose input is always negative never update. Leaky ReLU and GELU are common fixes.

Loss functions

The loss function measures how far predictions are from targets. Mean squared error is used for regression. Cross-entropy loss is used for classification and is derived from maximum likelihood: minimising cross-entropy is the same as maximising the log probability the model assigns to the correct class. Softmax converts raw logits into a probability distribution before cross-entropy is applied.

Gradient descent

Gradient descent updates every parameter in the direction that most reduces the loss: $w \leftarrow w - \eta \nabla L(w)$, where η is the learning rate. Full-batch gradient descent uses the whole dataset per step; stochastic gradient descent (SGD) uses one example; mini-batch SGD uses a small batch (32–512) and is the practical default because it balances noisy but cheap updates against stable but expensive ones.

A learning rate that is too high makes training diverge; too low and it crawls. Learning rate schedules (step decay, cosine annealing, warm-up) change η over time. Momentum accumulates an exponentially weighted average of past gradients to damp oscillation. Adam combines momentum with a per-parameter adaptive learning rate based on the running variance of gradients, and is the most common optimiser for deep networks.

Backpropagation

Backpropagation is the algorithm that computes the gradient of the loss with respect to every weight in the network. It applies the chain rule backwards, layer by layer: the gradient at a layer's output is multiplied by the local derivative of that layer to get the gradient at its input, which becomes the output gradient of the layer before it.

Key points:

- The forward pass stores each layer's activations because they are needed to compute local derivatives on the backward pass. This is why training uses far more memory than inference.
- Backpropagation is not an optimiser. It only produces gradients; gradient descent (or Adam, etc.) is what actually changes the weights.
- The cost of a backward pass is roughly twice the cost of the forward pass.
- Vanishing gradients happen when many small local derivatives (e.g. from saturated sigmoids) are multiplied together, so early layers receive almost no learning signal. Exploding gradients are the opposite; gradient clipping caps their norm.

A common misconception is that backpropagation is specific to neural networks. It is reverse-mode automatic differentiation applied to a computational graph, and the same machinery differentiates any program built from differentiable operations.

Weight initialisation

Weights are initialised randomly to break symmetry; if all weights started equal, every neuron in a layer would compute the same thing and receive the same gradient forever. Xavier (Glorot) initialisation scales weights by $1/\sqrt{\text{fan_in}}$ for tanh/sigmoid networks; He initialisation uses $2/\text{fan_in}$ and is preferred for ReLU networks, keeping activation variance roughly constant across layers.

Batch normalisation

Batch normalisation standardises each layer's pre-activations to zero mean and unit variance over the mini-batch, then applies a learned scale and shift. It lets you use higher learning rates, reduces sensitivity to initialisation and has a mild regularising effect. At inference time it uses running averages collected during training instead of batch statistics. Layer normalisation normalises across features for a single example instead, which is why it is used in transformers where batch statistics are awkward.